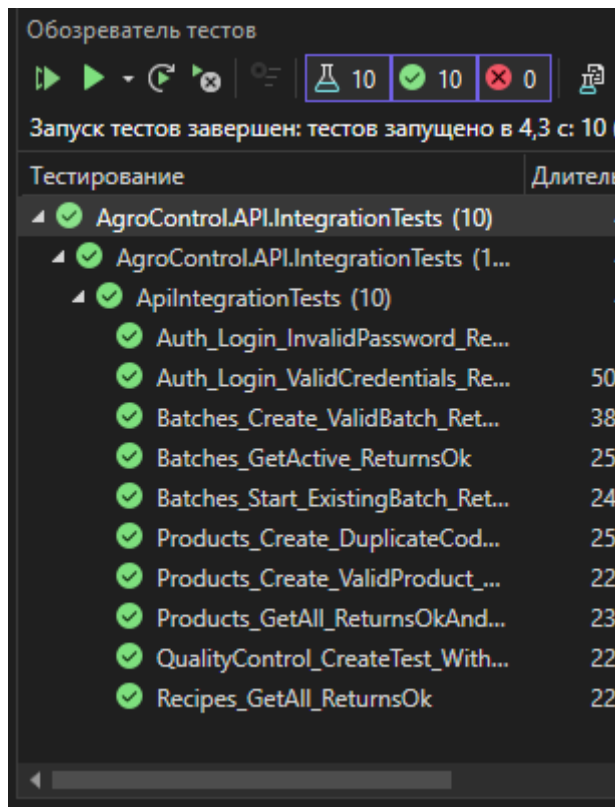




```
using Microsoft.VisualStudio.TestTools.UnitTesting;
```

```
using Newtonsoft.Json;
```

```
using System.Net;
```

```
using System.Net.Http.Json;
```

```
using System.Text;
```

```
using AgroControl.API.Models;
```

```
namespace AgroControl.API.IntegrationTests
```

```
{
```

```
    [TestClass]
```

```
    public class ApiIntegrationTests
```

```
    {
```

```
        private CustomWebApplicationFactory _factory;
```

```
        private HttpClient _client;
```

```
        [TestInitialize]
```

```
        public void Setup()
```

```
{  
    _factory = new CustomWebApplicationFactory();  
    _factory.Initialize();  
    _client = _factory.Client;  
}
```

```
[TestCleanup]  
  
public void Cleanup()  
{  
    _factory.Dispose();  
}
```

```
[TestMethod]  
  
public async Task Auth_Login_ValidCredentials_ReturnsOk()  
{  
    var loginData = new { Username = "tech.ivanov", Password = "Password123!" };  
    var response = await _client.PostAsJsonAsync("/api/Auth/login", loginData);  
    Assert.AreEqual(HttpStatusCode.OK, response.StatusCode);  
    var json = await response.Content.ReadAsStringAsync();  
    dynamic result = JsonConvert.DeserializeObject(json);  
    Assert.IsTrue((bool)result.success);  
}
```

```
[TestMethod]  
  
public async Task Auth_Login_InvalidPassword_ReturnsUnauthorized()  
{  
    var loginData = new { Username = "tech.ivanov", Password = "wrong" };  
    var response = await _client.PostAsJsonAsync("/api/Auth/login", loginData);  
    Assert.AreEqual(HttpStatusCode.Unauthorized, response.StatusCode);  
}
```

```
}
```

```
[TestMethod]
```

```
public async Task Products_GetAll_ReturnsOkAndList()
```

```
{
```

```
    var response = await _client.GetAsync("/api/Products");
```

```
    Assert.AreEqual(HttpStatusCode.OK, response.StatusCode);
```

```
    var json = await response.Content.ReadAsStringAsync();
```

```
    dynamic result = JsonConvert.DeserializeObject(json);
```

```
    Assert.IsTrue((bool)result.success);
```

```
    Assert.IsNotNull(result.data);
```

```
}
```

```
[TestMethod]
```

```
public async Task Products_Create_ValidProduct_ReturnsOk()
```

```
{
```

```
    var newProduct = new Product { Код = "P002", Наименование = "Инсектицид", Тип =  
"жидкий", Статус = "черновик" };
```

```
    var response = await _client.PostAsJsonAsync("/api/Products", newProduct);
```

```
    Assert.AreEqual(HttpStatusCode.OK, response.StatusCode);
```

```
    var json = await response.Content.ReadAsStringAsync();
```

```
    dynamic result = JsonConvert.DeserializeObject(json);
```

```
    Assert.IsNotNull(result, "Результат не может быть null");
```

```
    Assert.IsTrue((bool)result.success);
```

```
    // Проверяем наличие data и её свойств
```

```
    if (result.data != null)
```

```
    {
```

```
        Assert.AreEqual("P002", result.data.Код.ToString());
```

```
    }
```

```
}
```

```
[TestMethod]
```

```
public async Task Products_Create_DuplicateCode_ReturnsBadRequest()
```

```
{
```

```
    var duplicateProduct = new Product { Код = "P001", Наименование = "Дубликат" };
```

```
    var response = await _client.PostAsJsonAsync("/api/Products", duplicateProduct);
```

```
    Assert.AreEqual(HttpStatusCode.BadRequest, response.StatusCode);
```

```
    var json = await response.Content.ReadAsStringAsync();
```

```
    dynamic result = JsonConvert.DeserializeObject(json);
```

```
    Assert.IsFalse((bool)result.success);
```

```
    Assert.IsTrue(result.message.ToString().Contains("таким кодом уже существует"));
```

```
}
```

```
[TestMethod]
```

```
public async Task Batches_GetActive_ReturnsOk()
```

```
{
```

```
    var response = await _client.GetAsync("/api/Batches/active");
```

```
    Assert.AreEqual(HttpStatusCode.OK, response.StatusCode);
```

```
    var json = await response.Content.ReadAsStringAsync();
```

```
    dynamic result = JsonConvert.DeserializeObject(json);
```

```
    Assert.IsTrue((bool)result.success);
```

```
}
```

```
[TestMethod]
```

```
public async Task Batches_Create_ValidBatch_ReturnsOk()
```

```
{
```

```
    var newBatch = new ProductionBatch
```

```
{
```

```

        НомерПартии = "B-INT-001",

        ЗаказID = 1,

        Статус = "запланирована"

    };

    var response = await _client.PostAsJsonAsync("/api/Batches", newBatch);

    Assert.AreEqual(HttpStatusCode.OK, response.StatusCode);

    var json = await response.Content.ReadAsStringAsync();

    dynamic result = JsonConvert.DeserializeObject(json);

    Assert.IsTrue((bool)result.success);

    Assert.IsNotNull(result.data.ID);
}

```

```

[TestMethod]

public async Task Batches_Start_ExistingBatch_ReturnsOk()

{

    // Создаём партию

    var createResponse = await _client.PostAsJsonAsync("/api/Batches", new
ProductionBatch

    {

        НомерПартии = "B-INT-START",

        ЗаказID = 1,

        Статус = "запланирована"

    });

    Assert.AreEqual(HttpStatusCode.OK, createResponse.StatusCode);

    var createJson = await createResponse.Content.ReadAsStringAsync();

    dynamic createResult = JsonConvert.DeserializeObject(createJson);

    Assert.IsNotNull(createResult, "Результат создания не может быть null");

    Assert.IsTrue((bool)createResult.success, "Создание партии не удалось");

    int batchId = createResult.data.ID;
}

```

```

var response = await _client.PostAsync($"/api/Batches/{batchId}/start", null);
Assert.AreEqual(HttpStatusCode.OK, response.StatusCode);
var json = await response.Content.ReadAsStringAsync();
dynamic result = JsonConvert.DeserializeObject(json);
Assert.IsTrue((bool)result.success);
}

```

[TestMethod]

```

public async Task QualityControl_CreateTest_WithoutBatch_ReturnsBadRequest()
{
    var test = new LabTest
    {
        НаименованиеПараметра = "Влажность",
        ИзмеренноеЗначение = "5.2"
    };

    var response = await _client.PostAsJsonAsync("/api/QualityControl", test);
    Assert.AreEqual(HttpStatusCode.BadRequest, response.StatusCode);
    var json = await response.Content.ReadAsStringAsync();
    dynamic result = JsonConvert.DeserializeObject(json);
    Assert.IsFalse((bool)result.success);
    Assert.IsTrue(result.message.ToString().Contains("Не указана партия"));
}

```

[TestMethod]

```

public async Task Recipes_GetAll_ReturnsOk()
{
    var response = await _client.GetAsync("/api/Recipes");
    Assert.AreEqual(HttpStatusCode.OK, response.StatusCode);
}

```

```
var json = await response.Content.ReadAsStringAsync();  
dynamic result = JsonConvert.DeserializeObject(json);  
Assert.IsTrue((bool)result.success);  
}  
}  
}
```